

Projet 8 — infrastructure de données

Construisez et testez une infrastructure de données

GreenCoop • Forecast 2.0

Contexte du projet

Qui ?

Entreprise : GreenAndCoop, un fournisseur coopératif français d'électricité d'origine renouvelable.

Objectif

Fournir quotidiennement des données météorologiques de qualité aux Data Scientists pour leurs modèles de prévision

Problématique

Business : Équilibrer le réseau, optimiser la production intermittente (solaire/éolien) et maîtriser les coûts d'achat sur le marché de gros.

Technique : Intégrer de nouvelles sources de données hétérogènes transmises à des fréquences variables (toutes les 10, 15 ou 30 minutes) dans un pipeline fiable et automatisé..

Données utilisées

Réseau InfoClimat : 4 stations (Bergues, Hazebrouck, Armentières, Lille-Lesquin).

Réseau Weather Underground : 2 stations amateurs (Ichtegem et La Madeleine).

Infrastructure Cible (AWS)

Ingestion : Airbyte.

Transformation & Tests : DBT.

Base de données : PostgreSQL hébergée sur AWS RDS.

Architecture : Pipeline ELT (Extract, Load, Transform).

Étape 0 – Préparez votre environnement de travail

Installation de Docker



Déployez une instance PostgreSQL avec Docker

 Image : postgres:15
 Port : 5433
 Volume : pgdata

```
docker compose up -d
```

Se connecter a la BDD



```
psql -h localhost -p 5433 -U ali_admin -d weather_data
```

(Ensuite taper le mdp)



Installation d'Airbyte

Taper dans le terminal :
abctl local install



Création de l'Environnement Virtuel Python (VENV)

Taper dans le terminal :
uv venv

Rentrer dans
l'environnement:
.venv\Scripts\activate



Installation de DBT

Taper dans le terminal :
uv pip install dbt-core dbt-postgres

Étape 1 - Récupérez les données météorologiques avec Airbyte

<http://localhost:8000>
(Site de airbyte)

→ Crée une destination →

Create a destination Postgres

Destination name

Connection

Host

Port

Database Name

Default Schema

Username

SSL Modes

SSH Tunnel Method

Optional fields

Password

Crée une source → Notification erreur

Create a source File (CSV, JSON, Excel, Feather, Parquet)

Source name

Dataset Name

File Format

Storage Provider

Optional fields

☐ User-Agent

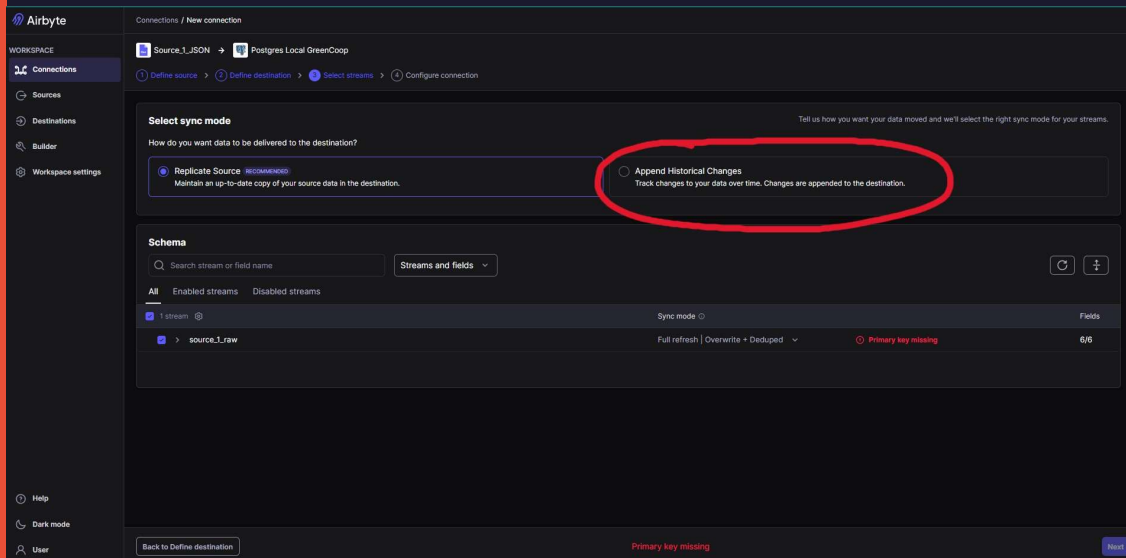
URL

Optional fields

[Fork in Builder](#) [Copy JSON](#) [Set up source](#)

Étape 1 - Récupérez les données météorologiques avec Airbyte

Crée une source



The screenshot shows the 'Select sync mode' step in the Airbyte interface. The 'Replicate Source' option is selected and highlighted with a red circle. Below it, the 'Schema' section shows a table with one stream named 'source_1_raw' and a 'Primary key missing' warning.

Select sync mode

Tell us how you want your data moved and we'll select the right sync mode for your streams.

How do you want data to be delivered to the destination?

☒ **Replicate Source** RECOMMENDED
Maintain an up-to-date copy of your source data in the destination.

☐ **Append Historical Changes**
Track changes to your data over time. Changes are appended to the destination.

Schema

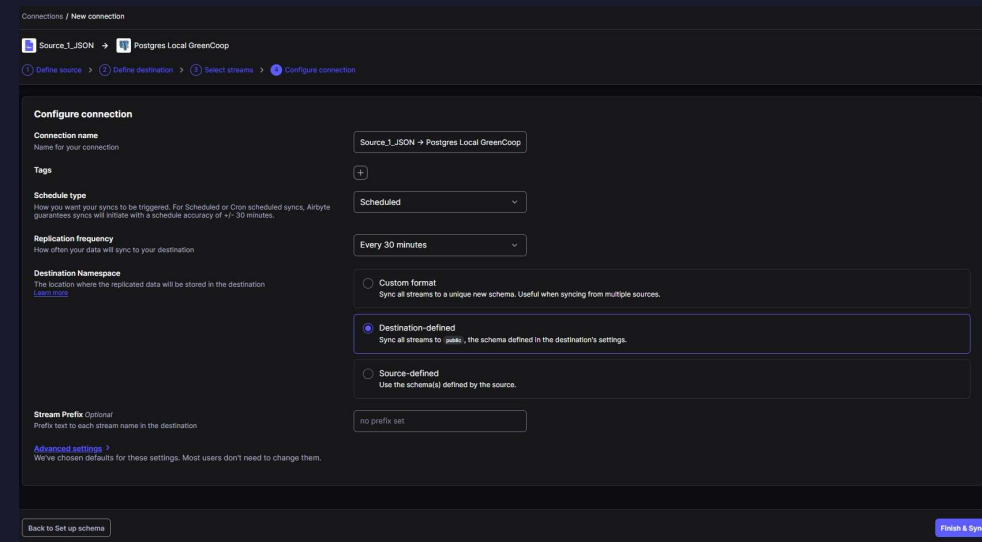
Search stream or field name Streams and fields

Stream	Sync mode	Fields
source_1_raw	Full refresh Overwrite + Deduped	6/6

Primary key missing

Back to Define destination Next

Consigne : les données brutes



The screenshot shows the 'Configure connection' step in the Airbyte interface. The 'Destination-defined' option is selected for the 'Destination Namespace'.

Configure connection

Connection name: Source_1_JSON -> Postgres Local GreenCoop

Tags: +

Schedule type: Scheduled

Replication frequency: Every 30 minutes

Destination Namespace: Destination-defined (Sync all streams to: postgres, the schema defined in the destination's settings.)

Stream Prefix: no prefix set

Advanced settings

Back to Set up schema Finish & Sync

Consigne : Données transmises toutes les 10, 15 ou 30 minutes

Étape 1 - Récupérez les données météorologiques avec Airbyte

Crée une source

Connections / Source_1_JSON → Postgres Local GreenCoop

Source_1_JSON → Postgres Local GreenCoop Every 30 minutes Cancel Sync ENABLED

Status Timeline Schema Mappings Settings

Active Streams Sync starting...

Status	Stream name	Latest sync in	records	Data fresh as of
Syncing	source_1_raw	Starting...		a few seconds ago

Connections / Source_1_JSON → Postgres Local GreenCoop

Source_1_JSON → Postgres Local GreenCoop Every 30 minutes Sync now ENABLED

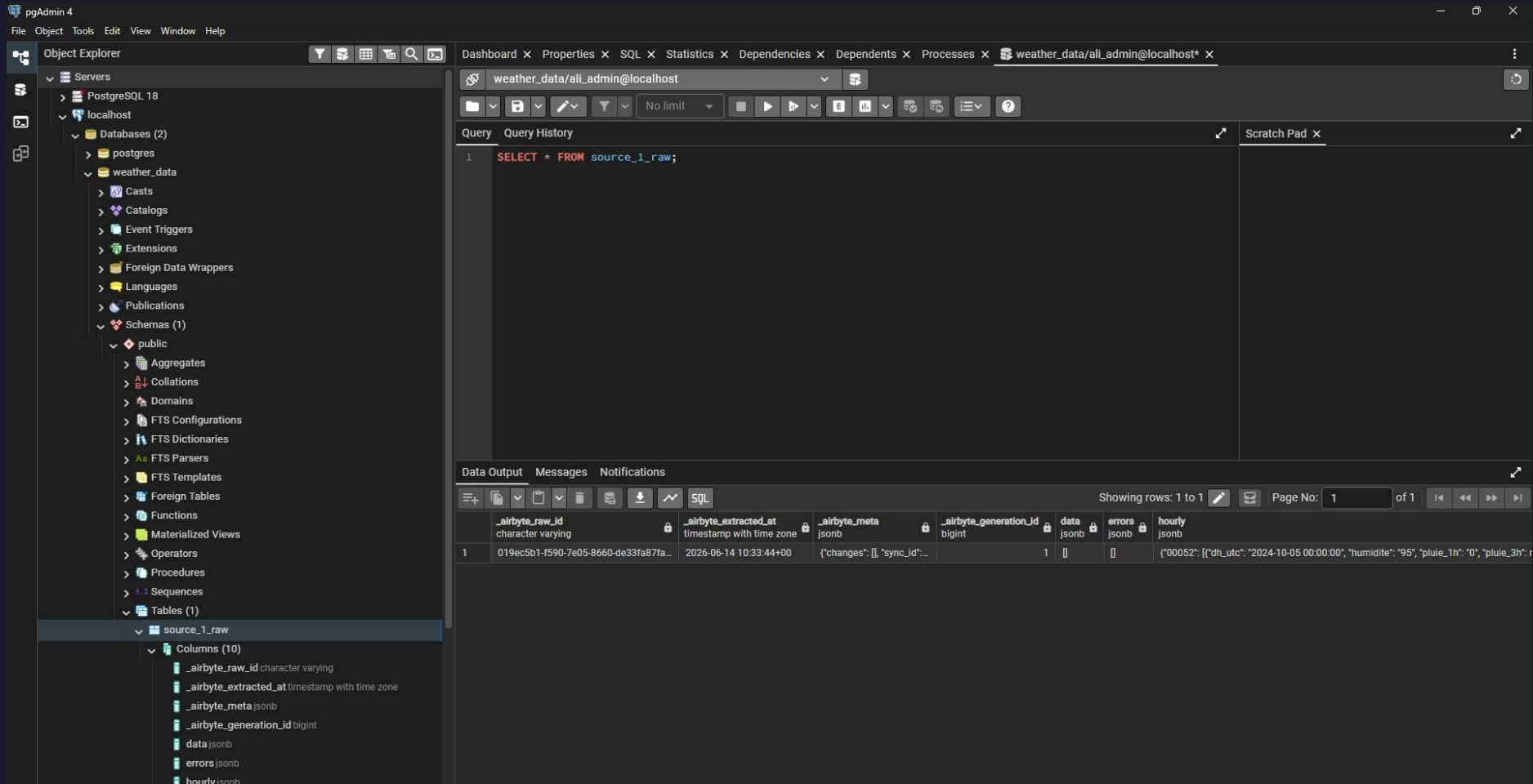
Status Timeline Schema Mappings Settings

Active Streams Next sync in 29 minutes

Status	Stream name	Latest sync in	records	Data fresh as of
Synced	source_1_raw	1 loaded		a few seconds ago

Étape 1 - Récupérez les données météorologiques avec Airbyte

Test de vérification



The screenshot shows the pgAdmin 4 interface. On the left, the Object Explorer displays the database structure, including the 'weather_data' database and the 'source_1_raw' table. The main pane shows a SQL query: `SELECT * FROM source_1_raw;`. The Data Output pane at the bottom displays the results of the query, showing a single row of data with columns: `_airbyte_raw_id`, `_airbyte_extracted_at`, `_airbyte_meta`, `_airbyte_generation_id`, `data`, `errors`, and `hourly`.

	<code>_airbyte_raw_id</code>	<code>_airbyte_extracted_at</code>	<code>_airbyte_meta</code>	<code>_airbyte_generation_id</code>	<code>data</code>	<code>errors</code>	<code>hourly</code>
1	019ec5b1-f590-7e05-8660-de33fa87fa...	2026-06-14 10:33:44+00	{'changes': [], 'sync_id':...	1	[]	[]	{'00052': {'dh_utc': '2024-10-05 00:00:00', 'humidite': '95', 'pluie_1h': '0', 'pluie_3h': r

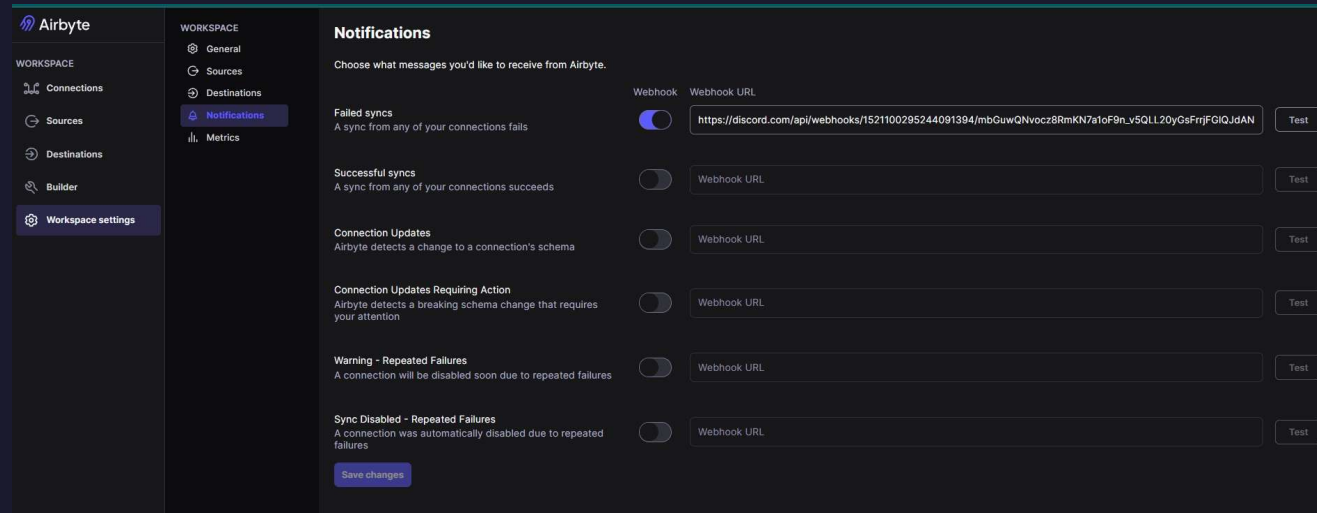
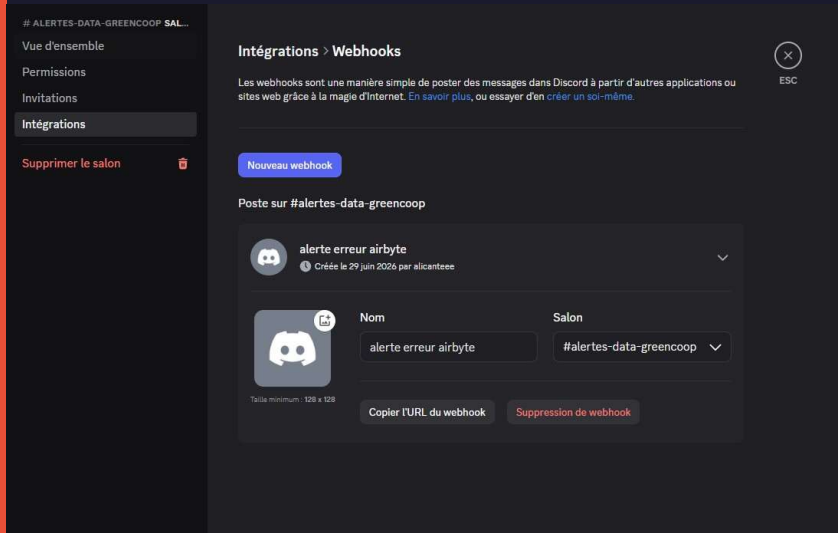
données brutes sale

Étape 1 - Récupérez les données météorologiques avec Airbyte

Notification erreur : sync fail

Création salon → modifier salon → intégration → webhook

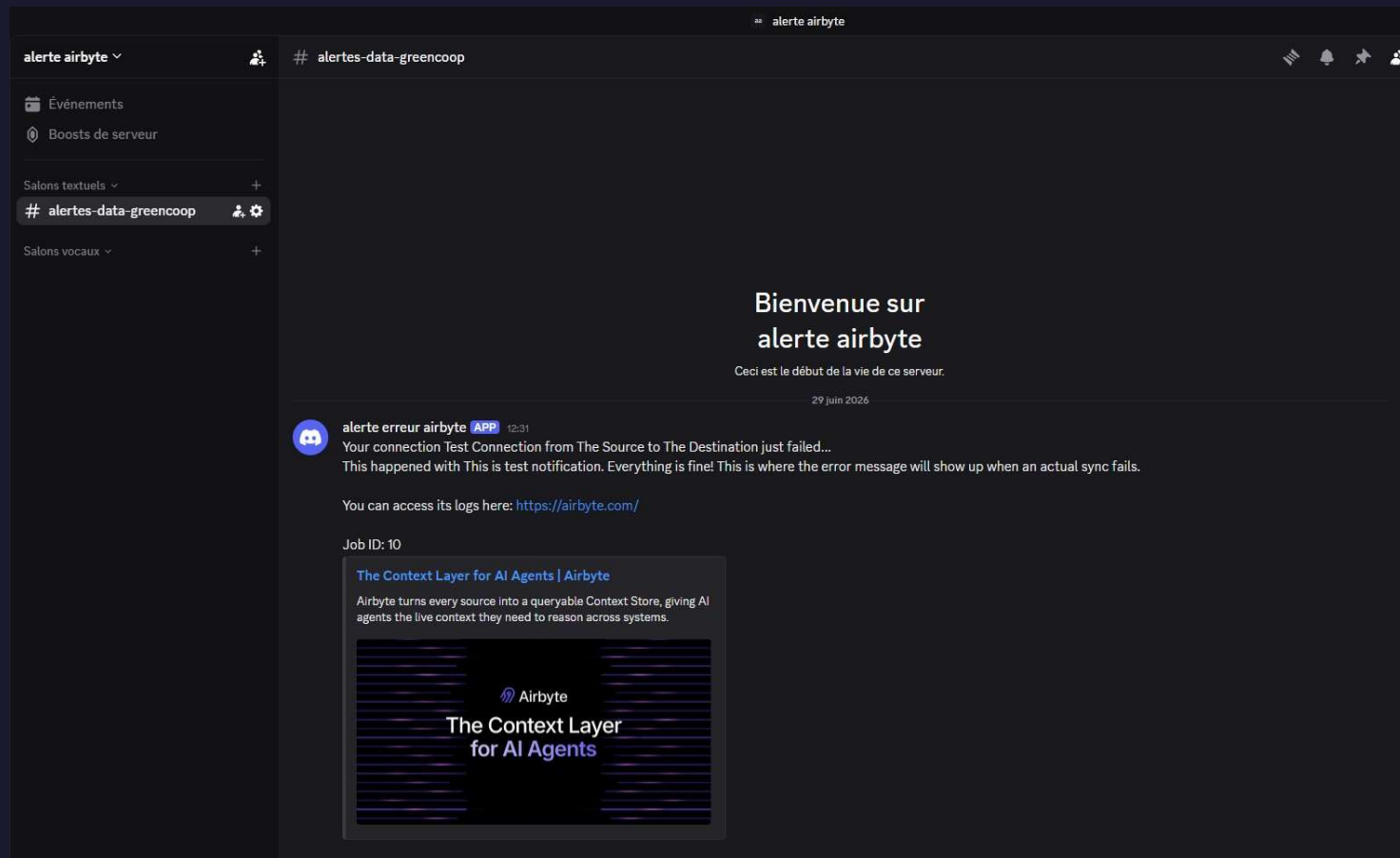
→ localhost:8000 → workspace settings → notification → webhook (activer) → colle URL webhook avec /slack à la fin



Étape 1 - Récupérez les données météorologiques avec Airbyte

Notification erreur : sync fail

Test de vérification :



Étape 2 – Transformez les données météorologiques avec DBT

Initialisation & Sources

Connexion à PostgreSQL
(profiles.yml)



Staging (Le Nettoyage) 🖌️

- Standardisation et typage des données (::numeric, ::timestamp)
- Aplatissement du format JSON (jsonb_array_elements)



Intermediate (L'Atelier) 🧩

- Harmonisation des colonnes manquantes (NULL::integer)
- Fusion des 3 sources hétérogènes (UNION ALL)



Marts (La Vitrine) 🌟

- Création du Schéma en étoile finale
- Injection des métadonnées statiques (Coordonnées GPS)

Étape 2 – Transformez les données météorologiques avec DBT

Initialisation & Sources

Connexion à PostgreSQL (profiles.yml)

```
forecast_project:
  target: dev
  outputs:

    dev:
      type: postgres
      host: localhost
      user: ali_admin
      password: "{{ env_var('DBT_PASSWORD') }}"
      port: 5433
      dbname: weather_data
      schema: public
      threads: 1
```

Définir comme variable pour caché le mdp :

set DBT_PASSWORD=ton_vrai_mot_de_passe_rds

Étape 2 – Transformez les données météorologiques avec DBT

Staging (Le Nettoyage) 🧹

standardisation, typage et aplatissement

(Extrait de code) :

```
WITH raw_data AS (  
  SELECT hourly  
  FROM {{ source('airbyte_raw', 'source_1_raw') }}  
)  
  
flattened_data AS (  
  SELECT  
  
    (reading->'id_station')::varchar AS station_id,  
    (reading->'dh_utc')::timestamp AS observation_date,  
    (reading->'temperature')::numeric AS temperature,  
  
  FROM raw_data,  
  jsonb_each(hourly) AS station(key, val),  
  jsonb_array_elements(station.val) AS reading  
)  
  
SELECT * FROM flattened_data
```

Exemple :

```
{  
  "STA_MADELEINE": [  
    {"dh_utc": "10:00", "temperature": 15},  
    {"dh_utc": "11:00", "temperature": 16}  
  ],  
  "STA_LILLE": [  
    {"dh_utc": "10:00", "temperature": 14}  
  ]  
}
```

key (L'ID de la station)	val (La boîte qui contient les relevés)
"STA_MADELEINE"	[{"dh_utc": "10:00", "temperature": 15}, {"dh_utc": "11:00", "temperature": 16}]
"STA_LILLE"	[{"dh_utc": "10:00", "temperature": 14}]

key	reading (Le relevé isolé)
"STA_MADELEINE"	{"dh_utc": "10:00", "temperature": 15}
"STA_MADELEINE"	{"dh_utc": "11:00", "temperature": 16}
"STA_LILLE"	{"dh_utc": "10:00", "temperature": 14}

station_id	observation_date	temperature
STA_MADELEINE	10:00	15
STA_MADELEINE	11:00	16
STA_LILLE	10:00	14

Étape 2 – Transformez les données météorologiques avec DBT

Staging (Le Nettoyage) 🧹

standardisation, typage et aplatissement

(Extrait de code) :

```
WITH raw_data AS (  
  SELECT stations  
  FROM {{ source('airbyte_raw', 'source_1_raw') }}  
)  
  
flattened_stations AS (  
  SELECT DISTINCT  
  
    (station_element->>'id')::varchar AS station_id,  
    (station_element->>'name')::varchar AS station_name,  
    (station_element->>'type')::varchar AS station_type,  
  FROM raw_data,  
  
  jsonb_array_elements(stations) AS station_element  
)  
  
SELECT * FROM flattened_stations
```

Exemple :

```
[  
  {"id": "STA_MADELEINE", "name": "La Madeleine", "type": "amateur"},  
  {"id": "STA_LILLE", "name": "Lille Centre", "type": "officiel"},  
  {"id": "STA_MADELEINE", "name": "La Madeleine", "type": "amateur"}  
]
```

station_element (La fiche isolée sur le bureau)

{"id": "STA_MADELEINE", "name": "La Madeleine", "type": "amateur"}

{"id": "STA_LILLE", "name": "Lille Centre", "type": "officiel"}

{"id": "STA_MADELEINE", "name": "La Madeleine", "type": "amateur"}

station_id	station_name	station_type
STA_MADELEINE	La Madeleine	amateur
STA_LILLE	Lille Centre	officiel
STA_MADELEINE	La Madeleine	amateur

station_id	station_name	station_type
STA_MADELEINE	La Madeleine	amateur
STA_LILLE	Lille Centre	officiel

Étape 2 – Transformez les données météorologiques avec DBT

Staging (Le Nettoyage) 🧹

standardisation, typage et aplatissement

(Extrait de code) :

```
WITH raw_data AS (  
  SELECT *  
  FROM {{ source('airbyte_raw', 'source_2_raw') }}  
)
```

```
cleaned_data AS (  
  SELECT  
    'STA_MADELEINE' AS station_id,  
    "Time"::time AS observation_time,  
    ROUND((SUBSTRING("Temperature" FROM '[-0-9.]+')::numeric - 32) * 5.0/9.0, 1) AS temperature,  
    SUBSTRING("Humidity" FROM '[-0-9.]+')::numeric AS humidity,  
  FROM raw_data  
)  
SELECT * FROM cleaned_data
```

station_id	"Time"	"Temperature"	humidity
STA_MADELEINE	10:00 AM	59.0 °F	80
STA_MADELEINE	11:00 AM	60.8 °F	78

La conversion mathématique (Fahrenheit -> Celsius + Heure propre)

Exemple :

"Time"	"Temperature"	"Humidity"
10:00 AM	59.0 °F	80 %
11:00 AM	60.8 °F	78 %

station_id	observation_time	temperature	humidity
STA_MADELEINE	10:00:00	15.0	80
STA_MADELEINE	11:00:00	16.0	78

Étape 2 – Transformez les données météorologiques avec DBT

Staging (Le Nettoyage) 🖌️

Résultat des tests

```
(Projet 8) C:\Users\danie\Desktop\Projet 8\forecast_project>dbt run --select stg_source_1
11:41:22 Running with dbt=1.11.11
11:41:22 Registered adapter: postgres=1.10.0
11:41:22 Unable to do partial parsing because saved manifest not found. Starting full parse.
11:41:23 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.forecast_project.example
11:41:23 Found 1 model, 3 sources, 475 macros
11:41:23
11:41:23 Concurrency: 1 threads (target='dev')
11:41:23
11:41:24 1 of 1 START sql view model public.stg_source_1 ..... [RUN]
11:41:24 1 of 1 OK created sql view model public.stg_source_1 ..... [CREATE VIEW in 0.09s]
11:41:24
11:41:24 Finished running 1 view model in 0 hours 0 minutes and 0.38 seconds (0.38s).
11:41:24
11:41:24 Completed successfully
11:41:24
11:41:24 Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=1
```

Étape 2 – Transformez les données météorologiques avec DBT

Intermediate (L'Atelier)

(Extrait de code) :

```
WITH source_1 AS (  
  SELECT  
    station_id,  
    observation_date AS observation_time,  
    temperature::numeric,  
    NULL::numeric AS humidity  
  FROM {{ ref('stg_source_1') }}  
)  
source_2 AS (  
  SELECT station_id, observation_time, temperature, humidity  
  FROM {{ ref('stg_source_2') }}  
)  
SELECT * FROM source_1  
UNION ALL  
SELECT * FROM source_2
```

Exemple :

station_id	observation_date	temperature
STA_MADELEINE	10:00	15

station_id	observation_time	temperature	humidity
STA_MADELEINE	10:00:00	15.0	80
STA_LILLE	11:00:00	16.0	78

station_id	observation_time	temperature	humidity
STA_MADELEINE	10:00:00	15.0	NULL
STA_MADELEINE	10:00:00	15.0	80
STA_LILLE	11:00:00	16.0	78

Étape 2 – Transformez les données météorologiques avec DBT

Intermediate (L'Atelier)

Résultat des tests

```
(Projet 8) C:\Users\danie\Desktop\Projet 8\forecast_project>dbt run --select int_weather_data_unioned
13:54:37 Running with dbt=1.11.11
13:54:37 Registered adapter: postgres=1.10.0
13:54:38 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.forecast_project.example
13:54:38 Found 5 models, 3 sources, 475 macros
13:54:38
13:54:38 Concurrency: 1 threads (target='dev')
13:54:38
13:54:39 1 of 1 START sql view model public.int_weather_data_unioned ..... [RUN]
13:54:39 1 of 1 OK created sql view model public.int_weather_data_unioned ..... [CREATE VIEW in 0.08s]
13:54:39
13:54:39 Finished running 1 view model in 0 hours 0 minutes and 0.84 seconds (0.84s).
13:54:39
13:54:39 Completed successfully
13:54:39
13:54:39 Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=1
```

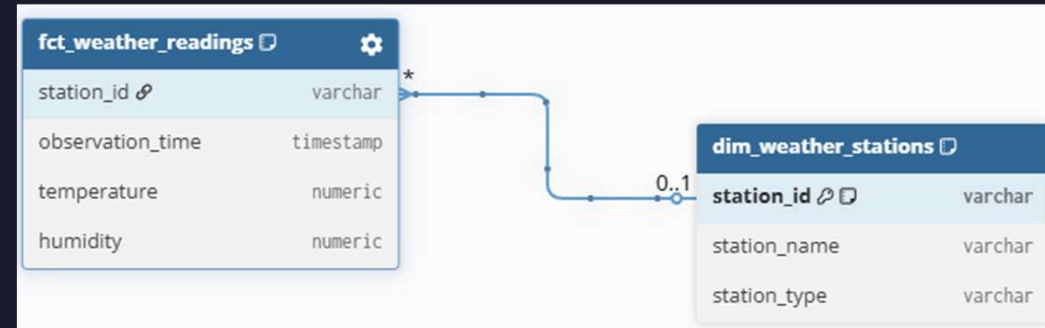
Étape 3 – Optimisez et sécurisez le schéma PostgreSQL

Marts (La Vitrine) 🌸

(Extrait de code) :

```
-- Fait : fct_weather_readings
{{ config(
    materialized='table',
    indexes=[{'columns': ['station_id']}, {'columns':
['observation_timestamp']}]
) }}

-- Dimension : dim_weather_stations
{{ config(
    materialized='table',
    indexes=[{'columns': ['station_id'], 'unique': True}]
) }}
```



Étape 3 – Optimisez et sécurisez le schéma PostgreSQL

Marts (La Vitrine) 🌟

Résultat des tests

```
(Projet 8) C:\Users\danie\Desktop\Projet 8\forecast_project>dbt run --select marts
14:00:07 Running with dbt=1.11.11
14:00:07 Registered adapter: postgres=1.10.0
14:00:08 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.forecast_project.example
14:00:09 Found 7 models, 3 sources, 475 macros
14:00:09
14:00:09 Concurrency: 1 threads (target='dev')
14:00:09
14:00:09 1 of 2 START sql view model public.dim_weather_stations ..... [RUN]
14:00:09 1 of 2 OK created sql view model public.dim_weather_stations ..... [CREATE VIEW in 0.07s]
14:00:09 2 of 2 START sql view model public.fct_weather_readings ..... [RUN]
14:00:09 2 of 2 OK created sql view model public.fct_weather_readings ..... [CREATE VIEW in 0.03s]
14:00:09
14:00:09 Finished running 2 view models in 0 hours 0 minutes and 0.38 seconds (0.38s).
14:00:09
14:00:09 Completed successfully
14:00:09
14:00:09 Done. PASS=2 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=2
```

Étape 4 – Tests de qualité et la documentation des données

Teste standard et métier

1. Tests de Structure (Standard DBT)

Ces tests garantissent que l'architecture de la base est saine et sans erreurs techniques.

- **Contrôle d'unicité** : Garantit qu'aucun doublon n'est présent dans la liste des stations.
- **Contrôle de complétude (Identifiants)** : Assure que chaque station possède un ID valide et renseigné.
- **Contrôle de complétude (Horodatage)** : Vérifie qu'aucun relevé météorologique ne manque de date ou d'heure.
- **Contrôle d'intégrité référentielle** : Assure la cohérence entre les faits et les dimensions (chaque mesure est rattachée à une station existante).

2. Test de Logique Métier (Custom DBT)

Ce test garantit que les données respectent les lois physiques du monde réel.

- **Contrôle de cohérence physique** : Vérifie que les données météorologiques (températures, humidité, vent) restent dans des limites physiquement possibles, afin d'exclure toute erreur de capteur matériel.

Étape 4 – Tests de qualité et la documentation des données

Teste standard et métier

```
(Projet 8) C:\Users\danie\Desktop\Projet 8\forecast_project>dbt test
13:25:36 Running with dbt=1.11.11
13:25:37 Registered adapter: postgres=1.10.0
13:25:37 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.forecast_project.example
13:25:37 Found 7 models, 6 data tests, 3 sources, 475 macros
13:25:37
13:25:37 Concurrency: 1 threads (target='dev')
13:25:37
13:25:37 1 of 6 START test not_null_dim_weather_stations_station_id ..... [RUN]
13:25:38 1 of 6 PASS not_null_dim_weather_stations_station_id ..... [PASS in 0.04s]
13:25:38 2 of 6 START test not_null_fct_weather_readings_observation_timestamp ..... [RUN]
13:25:38 2 of 6 PASS not_null_fct_weather_readings_observation_timestamp ..... [PASS in 0.02s]
13:25:38 3 of 6 START test not_null_fct_weather_readings_station_id ..... [RUN]
13:25:38 3 of 6 PASS not_null_fct_weather_readings_station_id ..... [PASS in 0.02s]
13:25:38 4 of 6 START test relationships_fct_weather_readings_station_id__station_id__ref_dim_weather_stations_ [RUN]
13:25:38 4 of 6 PASS relationships_fct_weather_readings_station_id__station_id__ref_dim_weather_stations_ [PASS in 0.03s]
13:25:38 5 of 6 START test test_valeurs_meteo_aberrantes ..... [RUN]
13:25:38 5 of 6 PASS test_valeurs_meteo_aberrantes ..... [PASS in 0.03s]
13:25:38 6 of 6 START test unique_dim_weather_stations_station_id ..... [RUN]
13:25:38 6 of 6 PASS unique_dim_weather_stations_station_id ..... [PASS in 0.03s]
13:25:38
13:25:38 Finished running 6 data tests in 0 hours 0 minutes and 0.37 seconds (0.37s).
13:25:38
13:25:38 Completed successfully
13:25:38
13:25:38 Done. PASS=6 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=6
```

Étape 4 – Tests de qualité et la documentation des données

Documentation

Logigramme généré avec la commande : `dbt docs serve`

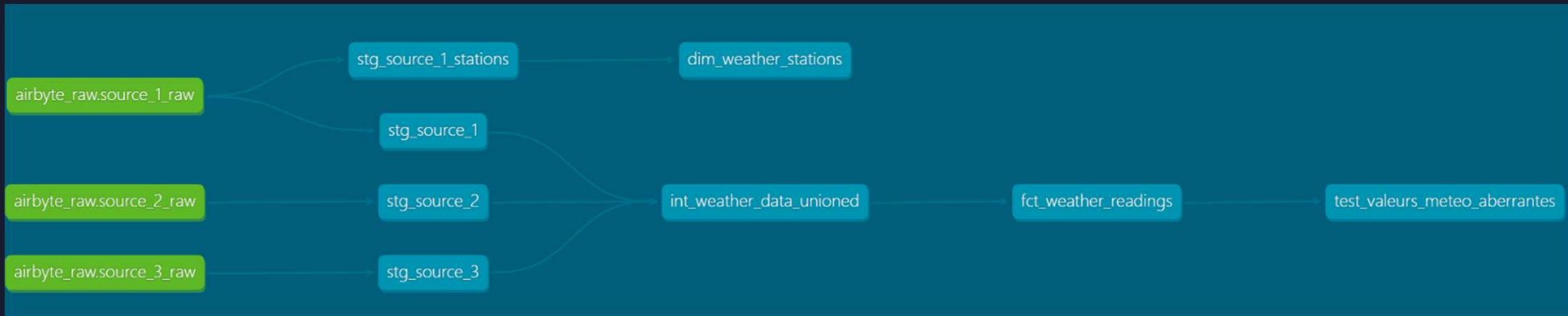


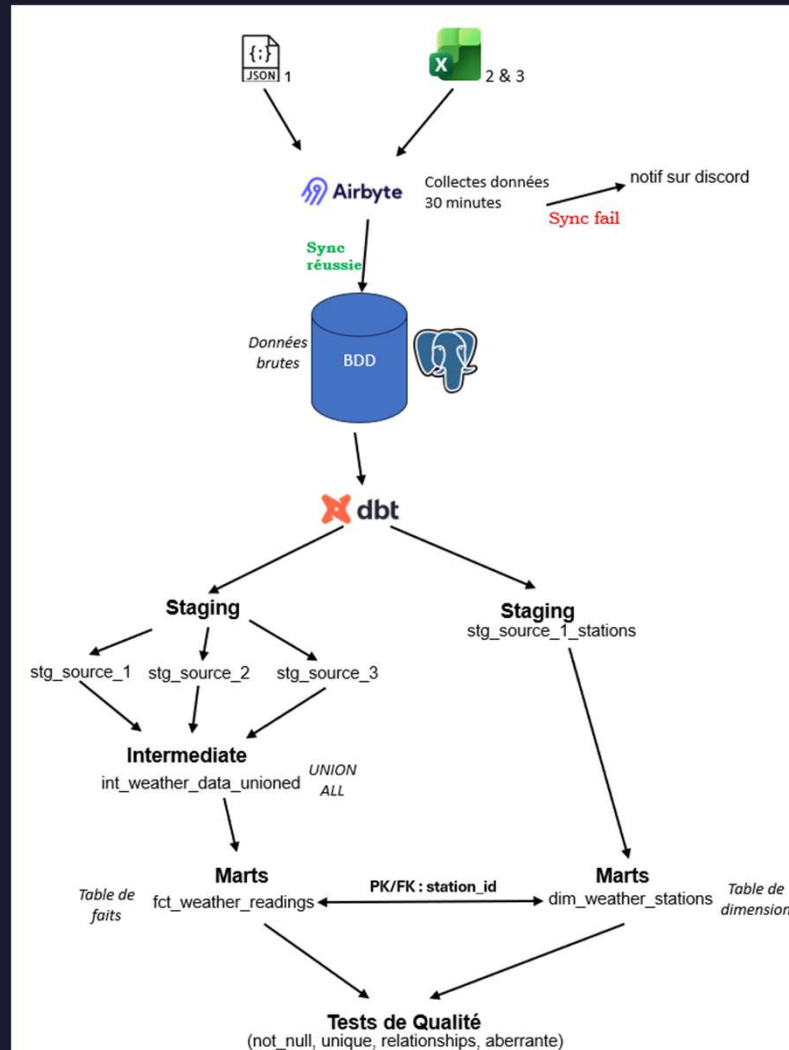
Schéma de la BDD



Étape 4 – Tests de qualité et la documentation des données

Documentation

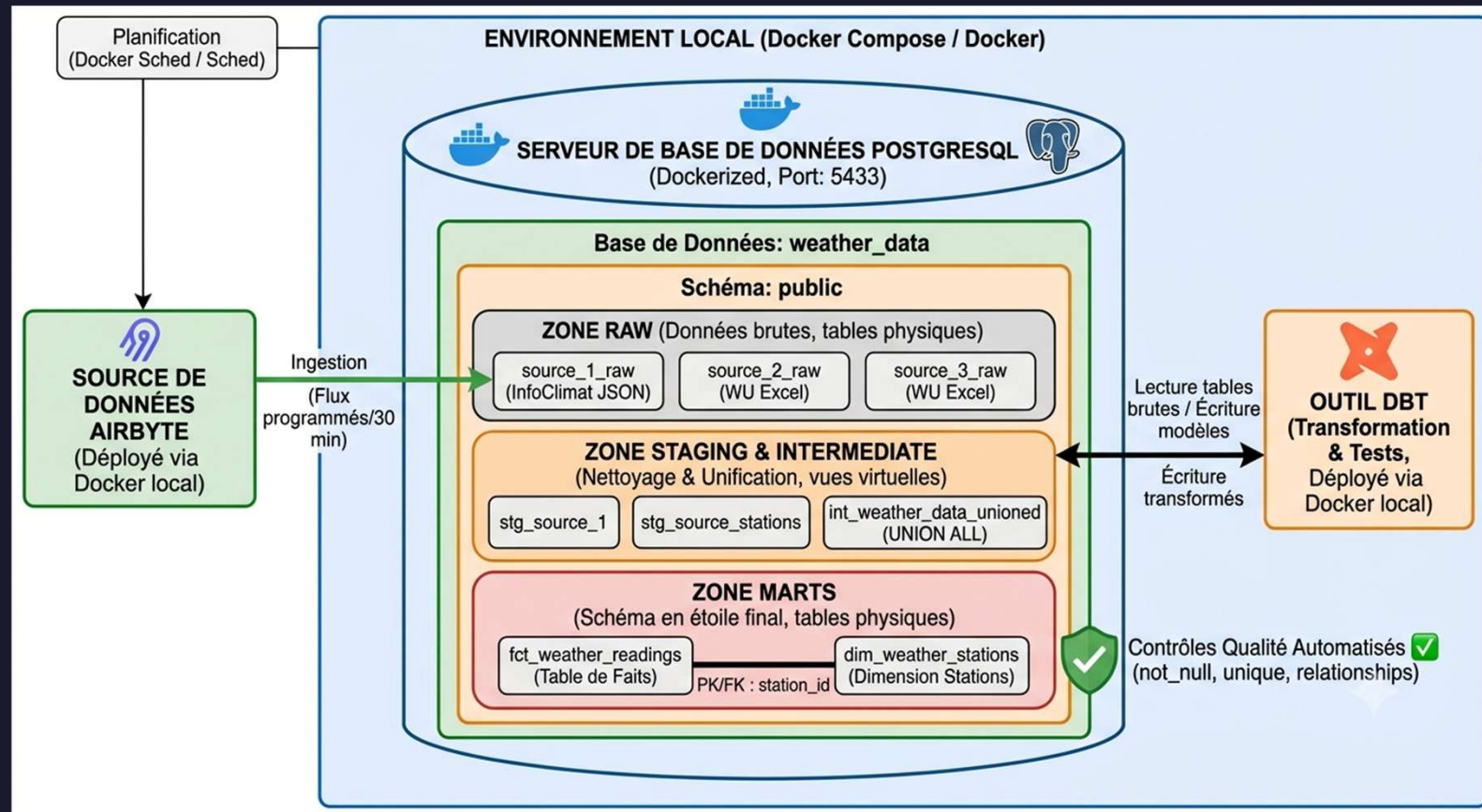
Logigramme complet



Étape 4 – Tests de qualité et la documentation des données

Documentation

Schéma de l'architecture de la base de données



Étape 4 – Tests de qualité et la documentation des données

Reporting sur temps d'accessibilité aux données

```
23 -- Quel est la température maximale par jour sur l'ensemble des stations ?
24 SELECT
25     DATE_TRUNC('day', observation_timestamp) AS jour,
26     MAX(temperature) AS temp_max,
27     MIN(temperature) AS temp_min
28 FROM fct_weather_readings
29 GROUP BY 1
30 ORDER BY 1 DESC;
```

Data Output Messages Notifications

	jour timestamp without time zone	temp_max numeric	temp_min numeric
1	2026-06-16 00:00:00	19.3	2.5
2	2024-10-07 00:00:00	18.6	11.3
3	2024-10-06 00:00:00	16.8	7.4
4	2024-10-05 00:00:00	17.8	2.7

Total rows: 4 Query complete 00:00:00.051

```
21 --Le taux d'Humidité Moyen par station et pourcentage
22 SELECT
23     s.station_name,
24     ROUND(AVG(f.humidity), 2) AS humidite_moyenne,
25     ROUND(AVG(f.humidity), 2) || '%' AS humidite_pourcentage
26 FROM fct_weather_readings f
27 JOIN dim_weather_stations s ON f.station_id = s.station_id
28 GROUP BY s.station_name;
```

Data Output Messages Notifications

	station_name character varying	humidite_moyenne numeric	humidite_pourcentage text
1	La Madeleine	84.87	84.87%
2	Lille-Lesquin	84.43	84.43%
3	Bergues	81.53	81.53%
4	WeerstationBS	78.16	78.16%
5	Armentières	84.20	84.20%
6	Hazebrouck	89.91	89.91%

Total rows: 6 Query complete 00:00:00.064

```
24 |-- Comparaison de la vitesse moyenne du vent par type de station
25 SELECT
26     s.station_type,
27     ROUND(AVG(f.wind_speed), 2) AS vent_moyen
28 FROM fct_weather_readings f
29 JOIN dim_weather_stations s ON f.station_id = s.station_id
30 GROUP BY s.station_type;
```

Data Output Messages Notifications

	station_type character varying	vent_moyen numeric
1	amateur	3.07
2	synop	13.92
3	static	2.54

Total rows: 3 Query complete 00:00:00.064

Étape 4 – Tests de qualité et la documentation des données

Reporting sur la qualité des données

Après

Staging : Standardisation et typage des sources brutes
Intermediate : Unification et harmonisation du pipeline"
Marts : Modélisation en Schéma en Étoile
Test : Test standard et métier

Staging, Intermediate, Marts et test qualité

Avant

Source 1 colonnes hourly (données météorologique) 15 000 lignes

hourly
jsonb
{ "00052": { "dh_utc": "2024-10-05 00:00:00", "humidite": "95", "pluie_1h": "0", "pluie_3h": null, "pression": "1019.1", "id_station": "00052", "vent_moyen": "0", "temperature": "6.4", "vent_r_...
{ "00052": { "dh_utc": "2024-10-05 00:00:00", "humidite": "95", "pluie_1h": "0", "pluie_3h": null, "pression": "1019.1", "id_station": "00052", "vent_moyen": "0", "temperature": "6.4", "vent_r_...
{ "00052": { "dh_utc": "2024-10-05 00:00:00", "humidite": "95", "pluie_1h": "0", "pluie_3h": null, "pression": "1019.1", "id_station": "00052", "vent_moyen": "0", "temperature": "6.4", "vent_r_...

Avant

Source 1 colonnes stations (informations stations) 50 lignes

stations
jsonb
{ "id": "00052", "name": "Armentières", "type": "static", "license": { "uri": "https://creativecommons.org/licenses/by/2.0/fr/", "source": "infoclimat.fr", "license": "CC BY", "metadonnees": "ht...
{ "id": "00052", "name": "Armentières", "type": "static", "license": { "uri": "https://creativecommons.org/licenses/by/2.0/fr/", "source": "infoclimat.fr", "license": "CC BY", "metadonnees": "ht...
{ "id": "00052", "name": "Armentières", "type": "static", "license": { "uri": "https://creativecommons.org/licenses/by/2.0/fr/", "source": "infoclimat.fr", "license": "CC BY", "metadonnees": "ht...

Avant

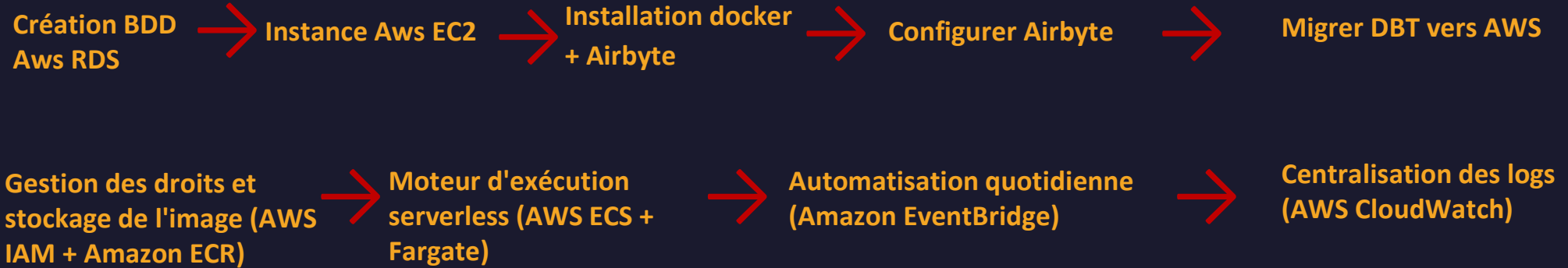
Source 2 colonnes stations (données météorologique)

UV	Gust	Time	Wind	Solar	Speed	Humidity	Pressure	Dew_Point	Temperature	Precip_Rate_	Precip_Accum_
numeric	character varying	character varying	character varying	character varying	character varying	character varying	character varying	character varying	character varying	character varying	character varying
0	10.4 mph	00:04:00	WSW	0 w/m²	8.2 mph	87 %	29.48 in	53.1 °F	56.8 °F	0.00 in	0.00 in
0	9.8 mph	00:09:00	WSW	0 w/m²	7.9 mph	87 %	29.47 in	52.9 °F	56.8 °F	0.00 in	0.00 in
0	12.8 mph	00:14:00	West	0 w/m²	10.3 mph	86 %	29.47 in	52.8 °F	57.0 °F	0.00 in	0.00 in

	station_id	observation_timestamp	temperature	humidity	pressure	wind_speed	wind_gust	wind_direction	rain_1h	rain_3h	dew_point
	character varying	timestamp without time zone	numeric	numeric	numeric	numeric	numeric	integer	numeric	numeric	numeric
1	00052	2024-10-05 00:00:00	6.4	95	1019.1	0	[null]	134	0	[null]	5.6
2	00052	2024-10-05 00:10:00	6.3	95	1019.1	0	[null]	134	[null]	[null]	5.6
3	00052	2024-10-05 00:20:00	6.3	96	1019	0	[null]	134	[null]	[null]	5.6
4	00052	2024-10-05 00:30:00	6.4	96	1019	0	[null]	134	[null]	[null]	6.1
5	00052	2024-10-05 00:40:00	6.3	95	1019	0	[null]	134	[null]	[null]	5.6
6	00052	2024-10-05 00:50:00	6.2	95	1018.9	0	[null]	134	[null]	[null]	5.6
7	00052	2024-10-05 01:00:00	6.1	95	1018.9	0	[null]	134	0	[null]	5.6
8	00052	2024-10-05 01:10:00	5.8	95	1018.8	0	[null]	134	[null]	[null]	5
9	00052	2024-10-05 01:20:00	5.4	95	1018.8	0	[null]	134	[null]	[null]	5
10	00052	2024-10-05 01:30:00	5.2	95	1018.7	0	[null]	134	[null]	[null]	4.4
11	00052	2024-10-05 01:40:00	5	95	1018.7	0	[null]	134	[null]	[null]	4.4
12	00052	2024-10-05 01:50:00	4.9	95	1018.7	0	[null]	134	[null]	[null]	4.4
13	00052	2024-10-05 02:00:00	4.8	96	1018.6	0	[null]	134	0	[null]	4.4
14	00052	2024-10-05 02:10:00	4.8	96	1018.6	0	[null]	134	[null]	[null]	4.4
15	00052	2024-10-05 02:20:00	4.8	96	1018.5	0	[null]	134	[null]	[null]	4.4
16	00052	2024-10-05 02:30:00	4.9	96	1018.5	0	[null]	134	[null]	[null]	4.4
17	00052	2024-10-05 02:40:00	4.8	96	1018.5	0	[null]	134	[null]	[null]	4.4
18	00052	2024-10-05 02:50:00	4.7	96	1018.5	0	[null]	134	[null]	[null]	3.9
19	00052	2024-10-05 03:00:00	4.6	96	1018.5	0	[null]	134	0	[null]	3.9
20	00052	2024-10-05 03:10:00	4.4	96	1018.4	0	[null]	134	[null]	[null]	3.9

	station_id	station_name	station_type	latitude	longitude	elevation
	character varying	character varying	character varying	numeric	numeric	integer
1	STATIC0010	Hazebrouck	static	50.734	2.545	31
2	00052	Armentières	static	50.689	2.877	16
3	07015	Lille-Lesquin	synop	50.575	3.092	47
4	000R5	Bergues	static	50.968	2.441	17
5	STA_MADELEINE	La Madeleine	amateur	50.659	3.07	23
6	STA_STATION_3	WeerstationBS	amateur	51.092	2.999	15

Étape 5 - Déployez sur AWS



Étape 5 - Déployez sur AWS

Création BDD Aws RDS

AWS → tape « RDS »
« Base de données »
« crée une base de données »

- 30 go
- Sauvegarde automatique

Instance Aws EC2

AWS → tape « EC2 »
« instance »
« lancer des instances »

- Amazon Linux 2023
- t2.xlarge

Installation docker + Airbyte

- Se Connecter à l'instance crée avec le terminal :
`ssh -i "cle-airbyte.pem" ec2-user@<IP_PUBLIQUE_AWS>`

- Installer docker :
`sudo yum install -y docker`

-Données le droit ADMIN à docker:
`sudo usermod -a -G docker ec2-user`

-Lancer docker :
`sudo systemctl start docker`

Activer Docker au démarrage automatique:
`sudo systemctl enable docker`

Quitter et relancer docker pour prendre en compte les changements:
`exit`
`ssh -i "cle-airbyte.pem" ec2-user@<IP_PUBLIQUE_AWS>`

Télécharger et installer l'outil abctl
`curl -LsfS https://get.airbyte.com | bash -`

Télécharger et installer airbyte via abctl
`sudo abctl local install`

Étape 5 - Déployez sur AWS

Configurer Airbyte

Afficher identifiant de connexion a Airbyte

```
abctl local credentials
```

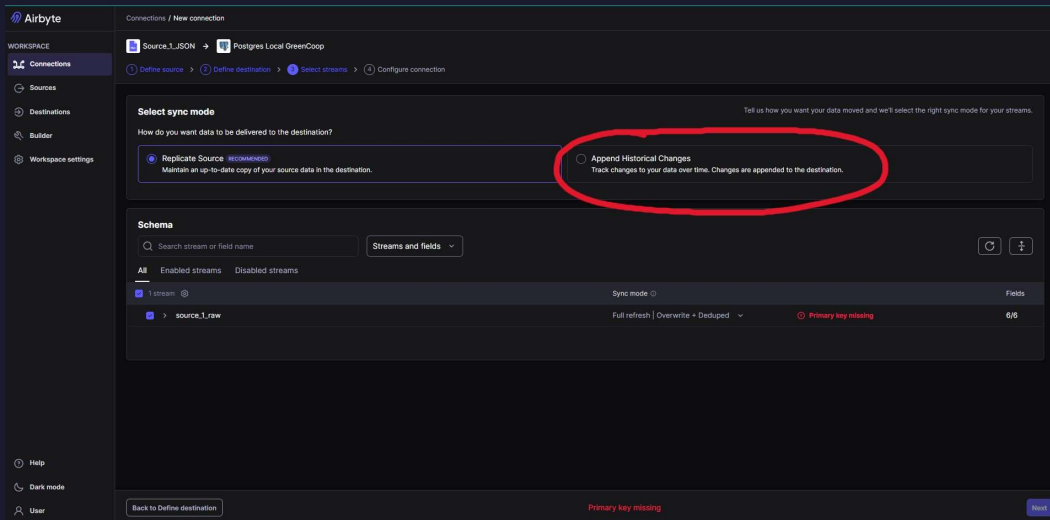
- Se Connecter à Airbyte et crée les source et destination:

The screenshot shows the 'Create a destination' form in Airbyte for a Postgres connection. The form is titled 'Create a destination' with a Postgres icon. It has a 'Destination name' field with the value 'Postgres Local GreenCoop'. Below this is the 'Connection' section with fields for 'Host' (host.docker.internal), 'Port' (5433), 'Database Name' (weather_data), 'Default Schema' (public), 'Username' (air_admin), 'SSL Modes' (disable), and 'SSH Tunnel Method' (No Tunnel). There is an 'Optional fields' section with a 'Password' field.

The screenshot shows the 'Create a source' form in Airbyte for a File (CSV, JSON, Excel, Feather, Parquet) connection. The form is titled 'Create a source' with a File icon. It has a 'Source name' field with the value 'Source_1_JSON'. Below this is the 'Dataset Name' field with the value 'source_1_raw'. The 'File Format' is set to 'json'. The 'Storage Provider' is set to 'HTTPS: Public Web'. There is an 'Optional fields' section with a 'User-Agent' field. The 'URL' field contains the value 'https://s3.eu-west-1.amazonaws.com/course.oc-static.com/projects/922_Data+Engineer/922_P8/Data_Source1_011024-07'. At the bottom, there are buttons for 'Fork in Builder', 'Copy JSON', and 'Set up source'.

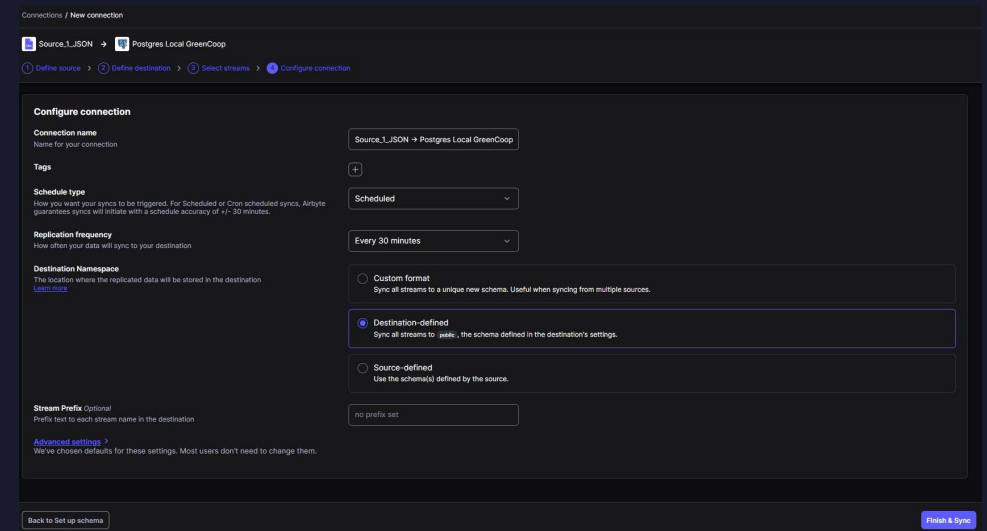
Étape 5 - Déployez sur AWS

Configurer Airbyte

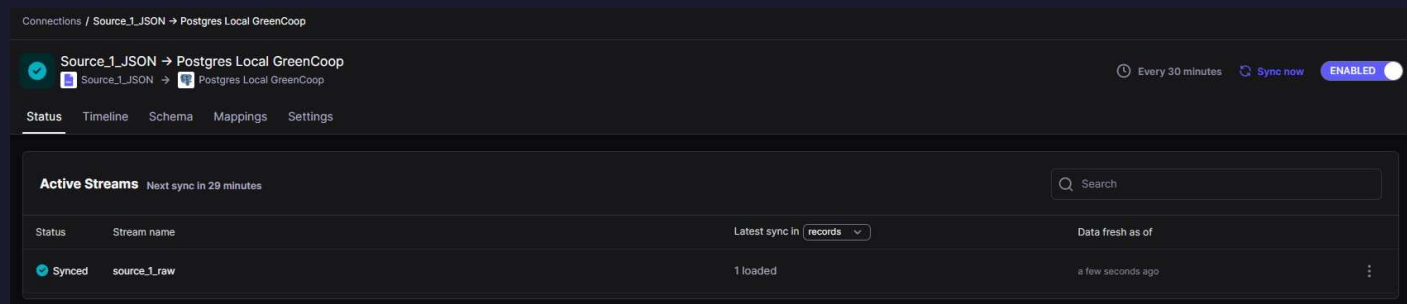


The screenshot shows the 'Select sync mode' step in the Airbyte interface. The 'Replicate Source' option is selected. The 'Append Historical Changes' option is circled in red. Below, the 'Schema' section shows a table with one stream, 'source_1_raw', which has a 'Primary key missing' error.

Stream	Sync mode	Fields
source_1_raw	Full refresh Overwrite + Deduped	6/6



The screenshot shows the 'Configure connection' step. The 'Connection name' is 'Source_1_JSON -> Postgres Local GreenCoop'. The 'Schedule type' is 'Scheduled'. The 'Replication frequency' is 'Every 30 minutes'. The 'Destination Namespace' is 'Custom format'. The 'Stream Prefix' is 'no prefix set'.



The screenshot shows the 'Status' page for the connection. The connection is 'ENABLED' and syncs 'Every 30 minutes'. The 'Active Streams' section shows one stream, 'source_1_raw', which is 'Synced' and has '1 loaded' records.

Status	Stream name	Latest sync in	Data fresh as of
Synced	source_1_raw	1 loaded	a few seconds ago

Étape 5 - Déployez sur AWS

Migrer DBT vers AWS

Connexion à PostgreSQL pour aws (profiles.yml)

```
aws:  
  type: postgres  
  host: database-2.cf44a0w06wh1.eu-west-3.rds.amazonaws.com  
  user: postgres  
  password: "{{ env_var('DBT_PASSWORD') }}"  
  port: 5432  
  dbname: weather_data_RDS  
  schema: public  
  threads: 1
```

Définir comme variable pour caché le mdp :

set DBT_PASSWORD=ton_vrai_mot_de_passe_rds

Étape 5 - Déployez sur AWS

Migrer DBT vers AWS

Connexion et configuration dbt sur aws :

- `dbt debug -t aws` : C'est le test de connexion (comme vérifier que tu as la bonne clé pour ouvrir la porte de ton RDS).
- `dbt run -t aws` : Ça prend tes requêtes SQL locales, ça les envoie à ton AWS RDS, et ça lui dit de créer physiquement tes tables finales (comme `fct_weather_readings`) directement dans le Cloud. C'est la construction de l'entrepôt.
- `dbt test -t aws` : Ça vérifie que les données qui viennent d'être créées sur AWS respectent tes règles de teste de qualité (teste standard et test metier).
- `dbt docs generate -t aws && dbt docs serve` : 100 % juste. Ça génère ton site web avec le logigramme (le lineage) mis à jour pour la cible de production.

Étape 5 - Déployez sur AWS

Migrer DBT vers AWS

Connection et configuration dbt sur aws :

dbt debug -t aws

```
(Projet 8) C:\Users\danie\Desktop\Projet 8\forecast_project>dbt debug -t aws
19:37:22 Running with dbt=1.11.11
19:37:22 dbt version: 1.11.11
19:37:22 python version: 3.13.5
19:37:22 python path: C:\Users\danie\Desktop\Projet 8\.venv\Scripts\python.exe
19:37:22 os info: Windows-11-10.0.26200-SP0
19:37:22 Using profiles dir at C:\Users\danie\.dbt
19:37:22 Using profiles.yml file at C:\Users\danie\.dbt\profiles.yml
19:37:22 Using dbt_project.yml file at C:\Users\danie\Desktop\Projet 8\forecast_project\dbt_project.yml
19:37:22 adapter type: postgres
19:37:22 adapter version: 1.10.0
19:37:23 Configuration:
19:37:23   profiles.yml file [OK found and valid]
19:37:23   dbt_project.yml file [OK found and valid]
19:37:23 Required dependencies:
19:37:23   - git [OK found]

19:37:23 Connection:
19:37:23   host: database-2.cf44a0w06wh1.eu-west-3.rds.amazonaws.com
19:37:23   port: 5432
19:37:23   user: postgres
19:37:23   database: weather_data_RDS
19:37:23   schema: public
19:37:23   connect_timeout: 10
19:37:23   role: None
19:37:23   search_path: None
19:37:23   keepalives_idle: 0
19:37:23   sslmode: None
19:37:23   sslcert: None
19:37:23   sslkey: None
19:37:23   sslrootcert: None
19:37:23   application_name: dbt
19:37:23   retries: 1
19:37:23 Registered adapter: postgres=1.10.0
19:37:23 Connection test: [OK connection ok]

19:37:23 All checks passed!
```

dbt run -t aws

```
(Projet 8) C:\Users\danie\Desktop\Projet 8\forecast_project>dbt run -t aws
19:41:12 Running with dbt=1.11.11
19:41:12 Registered adapter: postgres=1.10.0
19:41:12 Unable to do partial parsing because config vars, config profile, or config target have changed
19:41:12 Unable to do partial parsing because profile has changed
19:41:13 [WARNING][MissingArgumentsPropertyInGenericTestDeprecation]: Deprecated
functionality
Found top-level arguments to test 'relationships' defined on
'fct_weather_readings' in package 'forecast_project' (models\marts\schema.yml).
Arguments to generic tests should be nested under the 'arguments' property.
19:41:13 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.forecast_project.example
19:41:13 Found 7 models, 6 data tests, 3 sources, 475 macros
19:41:13
19:41:13 Concurrency: 1 threads (target='aws')
19:41:13
19:41:14 1 of 7 START sql view model public.stg_source_1 ..... [RUN]
19:41:14 1 of 7 OK created sql view model public.stg_source_1 ..... [CREATE VIEW in 0.19s]
19:41:14 2 of 7 START sql view model public.stg_source_1_stations ..... [RUN]
19:41:14 2 of 7 OK created sql view model public.stg_source_1_stations ..... [CREATE VIEW in 0.11s]
19:41:14 3 of 7 START sql view model public.stg_source_2 ..... [RUN]
19:41:14 3 of 7 OK created sql view model public.stg_source_2 ..... [CREATE VIEW in 0.11s]
19:41:14 4 of 7 START sql view model public.stg_source_3 ..... [RUN]
19:41:14 4 of 7 OK created sql view model public.stg_source_3 ..... [CREATE VIEW in 0.08s]
19:41:14 5 of 7 START sql table model public.dim_weather_stations ..... [RUN]
19:41:14 5 of 7 OK created sql table model public.dim_weather_stations ..... [SELECT 6 in 0.10s]
19:41:14 6 of 7 START sql view model public.int_weather_data_unioned ..... [RUN]
19:41:15 6 of 7 OK created sql view model public.int_weather_data_unioned ..... [CREATE VIEW in 0.11s]
19:41:15 7 of 7 START sql table model public.fct_weather_readings ..... [RUN]
19:41:15 7 of 7 OK created sql table model public.fct_weather_readings ..... [SELECT 4950 in 0.58s]
19:41:15
19:41:15 Finished running 2 table models, 5 view models in 0 hours 0 minutes and 1.78 seconds (1.78s).
19:41:15
19:41:15 Completed successfully
19:41:15
19:41:15 Done. PASS=7 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=7
19:41:15 [WARNING][DeprecationsSummary]: Deprecated functionality
Summary of encountered deprecations:
- MissingArgumentsPropertyInGenericTestDeprecation: 1 occurrence
To see all deprecation instances instead of just the first occurrence of each,
run command again with the '--show-all-deprecations' flag. You may also need to
run with '--no-partial-parse' as some deprecations are only encountered during
parsing.
```

Étape 5 - Déployez sur AWS

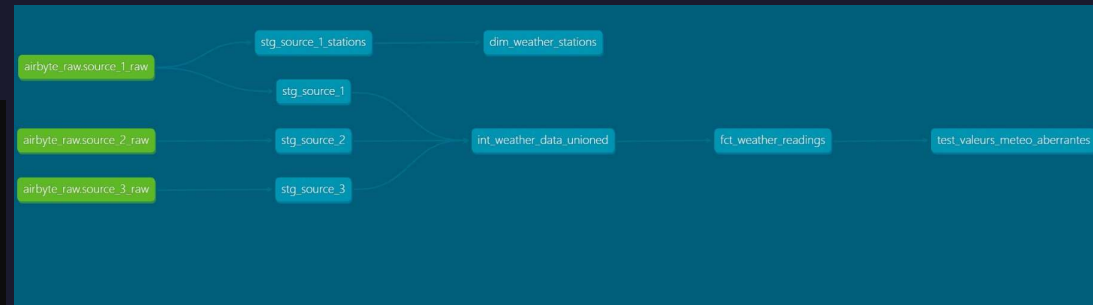
Migrer DBT vers AWS

Connection et configuration dbt sur aws :

dbt test -t aws

```
(Projet 8) C:\Users\danie\Desktop\Projet 8\forecast_project>dbt test -t aws
19:42:24 Running with dbt=1.11.11
19:42:24 Registered adapter: postgres=1.10.0
19:42:25 [WARNING]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
There are 1 unused configuration paths:
- models.forecast_project.example
19:42:25 Found 7 models, 6 data tests, 3 sources, 475 macros
19:42:25 Concurrency: 1 threads (target='aws')
19:42:25
19:42:25 1 of 6 START test not_null_dim_weather_stations_station_id ..... [RUN]
19:42:25 1 of 6 PASS not_null_dim_weather_stations_station_id ..... [PASS in 0.08s]
19:42:25 2 of 6 START test not_null_fct_weather_readings_observation_timestamp ..... [RUN]
19:42:25 2 of 6 PASS not_null_fct_weather_readings_observation_timestamp ..... [PASS in 0.06s]
19:42:25 3 of 6 START test not_null_fct_weather_readings_station_id ..... [RUN]
19:42:25 3 of 6 PASS not_null_fct_weather_readings_station_id ..... [PASS in 0.09s]
19:42:26 4 of 6 START test relationships_fct_weather_readings_station_id_station_id_ref_dim_weather_stations_ [RUN]
19:42:26 4 of 6 PASS relationships_fct_weather_readings_station_id_station_id_ref_dim_weather_stations_ [PASS in 0.10s]
19:42:26 5 of 6 START test test_valeurs_meteo_aberrantes ..... [RUN]
19:42:26 5 of 6 PASS test_valeurs_meteo_aberrantes ..... [PASS in 0.10s]
19:42:26 6 of 6 START test unique_dim_weather_stations_station_id ..... [RUN]
19:42:26 6 of 6 PASS unique_dim_weather_stations_station_id ..... [PASS in 0.08s]
19:42:26
19:42:26 Finished running 6 data tests in 0 hours 0 minutes and 0.94 seconds (0.94s).
19:42:26
19:42:26 Completed successfully
19:42:26
19:42:26 Done. PASS=6 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=6
```

dbt docs generate -t aws && dbt docs serve



The screenshot shows the dbt documentation interface. At the top is the dbt logo. Below it is a navigation bar with tabs for 'Project', 'Database' (which is selected), and 'Group'. The main content area is titled 'Tables and Views' and shows a tree structure under the 'public' schema. The listed items are: dim_weather_stations, fct_weather_readings, int_weather_data_unioned, source_1_raw, source_2_raw, source_3_raw, stg_source_1, stg_source_1_stations, stg_source_2, and stg_source_3.

Étape 5 - Déployez sur AWS

Gestion des droits AWS IAM

Clés d'accès (1)

Utilisez les clés d'accès pour effectuer des appels par programmation vers AWS à partir d'AWS CLI, des outils AWS pour PowerShell, des kits SDK AWS ou des appels d'API AWS directs. Vous pouvez disposer d'un maximum de deux clés d'accès (actives ou inactives) à la fois. [En savoir plus](#)

	Identifiant de la clé d'accès	Créé le	Dernière utilisation de la clé d'accès	Dernière région utilisée	Dernier service utilisé	Statut
☒	AKIA5PMI [redacted]	Il y a 8 jours	Il y a 8 jours	us-east-1	iam	Active

- Authentification locale via AWS CLI
- Application du principe du "Zero Trust".
- Aucun secret stocké en clair dans le code.

Pass pour mon ordi

Personne n'a accès sauf ceux qui ont la clé d'accès

(pas de partage de mdp)

Étape 5 - Déployez sur AWS

Stockage de l'image du projet via docker sur ECR

```
# On part d'une base Python officielle et légère
FROM python:3.10-slim
```

```
# On met à jour le système et on installe les dépendances requises pour faire tourner PostgreSQL
RUN apt-get update \
    && apt-get install -y --no-install-recommends git libpq-dev python3-dev build-essential \
    && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
```

```
# On installe la version exacte de dbt-postgres
RUN pip install --no-cache-dir dbt-core==1.8.2 dbt-postgres==1.8.2
```

```
# On crée le dossier de travail à l'intérieur du conteneur
WORKDIR /usr/app/dbt
```

```
# On copie tout ton code SQL et tes configurations dans le conteneur
COPY . .
```

```
# On copie le fichier profiles.yml dans le dossier caché que dbt utilise par défaut
RUN mkdir -p /root/.dbt
COPY profiles.yml /root/.dbt/profiles.yml
```

```
# La commande par défaut qui s'exécutera quand le conteneur s'allumera sur AWS
CMD ["dbt", "run", "-t", "aws"]
```

Étape 5 - Déployez sur AWS

Stockage de l'image du projet via docker sur ECR

dbt-forecast-repo

Résumé **Images** Stratégie de cycle de vie Autorisations Balises du référentiel

Images (3) [Infos](#)



Supprimer

Copier l'URI

Détails

Scanner

Afficher les commandes Push

Filtrer les images actives

☐	Balises d'image ▼	Type	Créé à ▼	Taille de l'image (Mo) ▼	Synthèse de l'image	Dernière extraction à ▼
☐	latest	Image Index	22 juin 2026, 13:43:50 (UTC+02)	233.61	sha256:5cbf6578d7c4b80e...	26 juin 2026, 15:56:05 (UTC+02)
☐	-	Image	22 juin 2026, 13:43:50 (UTC+02)	233.61	sha256:cc0fe07c913e4d24...	26 juin 2026, 15:56:05 (UTC+02)
☐	-	Image	22 juin 2026, 13:43:50 (UTC+02)	0.00	sha256:d490948d6678d5d...	-

Étape 5 - Déployez sur AWS

Exécution Serverless : AWS ECS & Fargate

Lancer un server qui s'allume juste pour exécute l'image docker et ce supprime

dbt-forecast-task:1

Dernière mise à jour à 1 juillet 2026, 11:50 (UTC+2:00) [Déployer](#) [Actions](#) [Créer une révision](#)

Présentation [Infos](#)

ARN [arn:aws:ecs:eu-west-1:123456789012:task-definition/dbt-forecast-task:1](#)

Rôle de la tâche -

Injection de pannes ☐ Désactivé

Statut ACTIVE

Rôle d'exécution de tâche [ecsTaskExecutionRole](#)

Heure de création 22 juin 2026, 14:04 (UTC+2:00)

Système d'exploitation/Architecture Linux/X86_64

Environnement de l'application Fargate

Mode réseau awsvpc

Conteneurs [JSON](#) [Placement de tâche](#) [Volumes \(0\)](#) [Requiert des attributs](#) [Balises](#)

Taille de la tâche

CPU de tâche unités S12 (0,5 vCPU)

Allocation maximale de CPU de tâche pour les conteneurs

CPU (unités)

Mémoire de tâche 1024 Mio (1 Gio)

Allocation maximale de mémoire de tâche pour la réservation de mémoire de conteneur

Mémoire (Mio)

▼ Conteneur : dbt-container [Infos](#)

[Détails](#) [JSON](#)

Image 926413416283.dkr.ecr.eu-west-3.amazonaws.com/dbt-forecast-repo:latest

Registre privé ☐ Désactivé

Processeur 0

Limite mémoire hard/soft -/-

Périphérique Neuron -

[<](#) [Environnement et secrets](#) [Paramètres réseau](#) [Sécurité et autorisations](#) [Cycle de vie et dépendances](#)

Variables d'environnement (1)

Cité	Type	Valeur
DBT_PASSWORD	value	

DBT_PASSWORD injectée en tant que variable d'environnement

Étape 5 - Déployez sur AWS

Exécution Serverless : AWS ECS & Fargate

Lancer un server qui s'allume juste pour exécute l'image docker et ce supprime

ecsTaskExecutionRole infos [Supprimer](#)

Récapitulatif [Modifier](#)

Date de création
June 22, 2026, 14:04 (UTC+02:00)

Dernière activité
🟢 il y a 4 jours

ARN
arn:aws:iam::920612345678:role/ecsTaskExecutionRole

Durée maximale de la session
1 heure

Autorisations Relations d'approbation Balises Dernier accès Révoquer les sessions

Politiques des autorisations (1) infos [Simuler i2](#) [Supprimer](#) [Ajouter des autorisations](#)

Vous pouvez attacher jusqu'à 10 politiques gérées.

Rechercher

Filtrer par Type
Tous les types

☐	Nom de la politique i2	Type	Entités attachées
☐	AmazonECSTaskExecutionRolePolicy	Gérées par AWS	1

AmazonECSTaskExecutionRolePolicy infos

Provides access to other AWS service resources that are required to run Amazon ECS tasks

Détails de la politique

Type
Gérées par AWS

Heure de création
November 16, 2017, 19:48 (UTC+01:00)

Heure de modification
November 16, 2017, 19:48 (UTC+01:00)

ARN
arn:aws:iam::920612345678:role/ecsTaskExecutionRole

Autorisations Entités attachées Versions de politique (1) Dernier accès

Autorisations définies dans cette politique infos [Récapitulatif](#) [JSON](#)

Les autorisations définies dans ce document de politique précisent les actions autorisées ou refusées. Afin de définir les autorisations d'une identité IAM (utilisateur, groupe de personnes ou rôle), attachez-lui une politique.

Rechercher

< Services Actions dans Elastic Container Registry (4 de 60)

Lecture (4 sur 25)

Action	Ressource	Demande de condition
BatchCheckLayerAvailability	Toutes ressources	None
BatchGetImage	Toutes ressources	None
GetAuthorizationToken	Toutes ressources	None
GetDownloadUrlForLayer	Toutes ressources	None

☐ Afficher les actions 56 restantes

4 actions que ECS fargate a le droit de faire

Étape 5 - Déployez sur AWS

Automatisation quotidienne (Amazon EventBridge)

AWS ECS & Fargate crée mais s'exécute quand ?

EventBridge Désactiver Modifier Supprimer

Détails de la planification

Nom de la planification EventBridge	Statut ✔ Activé	Heure de début de la planification -	Fenêtre horaire flexible -
Description -	ARN de planification arn:aws:scheduler:eu-west-1: [redacted] :schedule/default/EventBridge	Heure de fin de la planification -	Date de création Jun 22, 2026, 14:37:08 (UTC+02:00)
Nom du groupe de planifications default	Action après l'achèvement NONE	Fuseau horaire d'exécution Europe/Paris	Date de dernière modification Jul 01, 2026, 12:13:36 (UTC+02:00)

Planification **Cible** **Politique de nouvelles tentatives** **File d'attente de lettres mortes** **Chiffrement**

Cible Infos

Cible dbt-forecast-cluster i	ARN de la cible arn:aws:ecs:eu-west-3:926413416283:cluster/dbt-forecast-cluster	Rôle d'exécution Amazon_EventBridge_Scheduler_ECS_3cd0d8e886 i
Service Amazon ECS	API RunTask	
Paramètres supplémentaires ▼ EcsParameters TaskDefinitionArn: "arn:aws:ecs:eu-west-3:926413416283:task-definition/dbt-forecast-task:1", TaskCount: 1, LaunchType: "FARGATE",		

Planification

Fréquence fixe [Infos](#)

rate (1 days)

- Déclencheur planifié (Exécution toutes les 24h). (toutes les 24h)
- Orchestration automatisée de la tâche ECS Fargate. (tous se lance automatiquement)
- Mise en place d'un "Zero-Touch Pipeline". (pas d'intervention humaine)

Étape 5 - Déployez sur AWS

Exécution Serverless : AWS ECS & Fargate

AWS ECS & Fargate crée mais s'exécute quand ?

EventBridge

DésactiverModifierSupprimer

Détails de la planification

Nom de la planification
EventBridge

Description
-

Nom du groupe de planifications
default

Statut
✔ **Activé**

ARN de planification
arn:aws:scheduler:eu-west-1:**[redacted]**:schedule/default/EventBridge

Action après l'achèvement
NONE

Heure de début de la planification
-

Heure de fin de la planification
-

Fuseau horaire d'exécution
Europe/Paris

Fenêtre horaire flexible
-

Date de création
Jun 22, 2026, 14:37:08 (UTC+02:00)

Date de dernière modification
Jul 01, 2026, 12:13:36 (UTC+02:00)

Planification

Cible

Politique de nouvelles tentatives

File d'attente de lettres mortes

Chiffrement

Cible Infos

Cible
dbt-forecast-cluster [i](#)

Service
Amazon ECS

API
RunTask

ARN de la cible
arn:aws:ecs:eu-west-3:926413416283:cluster/dbt-forecast-cluster

Rôle d'exécution
[Amazon_EventBridge_Scheduler_ECS_3cd0d8e886](#) [i](#)

Paramètres supplémentaires
▼ **EcsParameters**

TaskDefinitionArn: "arn:aws:ecs:eu-west-3:926413416283:task-definition/dbt-forecast-task:1",
TaskCount: 1,
LaunchType: "FARGATE",

Planification

Fréquence fixe [Infos](#)

rate (1 days)

- Déclencheur planifié (Exécution toutes les 24h). (toutes les 24h)
- Orchestration automatisée de la tâche ECS Fargate. (tous se lance automatiquement)
- Mise en place d'un "Zero-Touch Pipeline". (pas d'intervention humaine)

11 Centralisation des logs (AWS CloudWatch)

Fargate détruit le conteneur dès que DBT a terminé. Sans CloudWatch, **tous les logs disparaissent dans le néant** et il est impossible de savoir si le pipeline a réussi ou échoué.

Comment ça fonctionne

- 1 EventBridge déclenche ECS**
Le “réveil” envoie le signal à ECS chaque matin
- 2 ECS lance le conteneur Fargate**
Le conteneur DBT démarre et exécute dbt run
- 3 Chaque ligne de log est interceptée**
La journalisation awslogs capture tout le texte du terminal DBT
- 4 Archivage dans CloudWatch**
Logs stockés dans le groupe /ecs/dbt-forecast-task, consultables à tout moment
- 5 Fargate détruit le conteneur**
Les logs restent dans CloudWatch même après la destruction du conteneur

 **Configuration** : lors de la création de la Task Definition ECS, l'option “Journalisation (awslogs)” était activée par défaut. AWS a automatiquement créé le Log Group /ecs/dbt-forecast-task dans CloudWatch. Zéro configuration supplémentaire nécessaire.

 **Valeur concrète** : si le pipeline plante à 4h du matin à cause d'une donnée météo corrompue, à 9h le Data Engineer ouvre CloudWatch, lit la ligne d'erreur exacte, identifie la source du problème et corrige. C'est l'observabilité — la preuve d'un déploiement cloud maîtrisé.

Étape 5 - Déployez sur AWS

Suivre la pipeline avec Aws CloudWatch

But identifier le bon trafic et les erreurs

Gestion des journaux > /ecs/dbt-forecast-task > ecs/dbt-container/96be8ec8e...

Événements de journaux

Vous pouvez utiliser la barre de filtre ci-dessous pour rechercher et faire correspondre des termes, des expressions ou des valeurs dans vos événements de journal. [En savoir plus sur les modèles de filtre](#)

Filter les événements : appuyez sur Entrée pour rechercher. Effacer 1m 30m 1h 12h Personnalisées Fuseau horaire UTC Affichage

Horodatage	Message
2026-06-26T13:56:24.652Z	@[0m13:56:24 Unable to do partial parsing because saved manifest not found. Starting full parse.
2026-06-26T13:56:28.479Z	@[0m13:56:28 [W33mWARNING@0m]: Deprecated functionality
2026-06-26T13:56:28.479Z	The 'tests' config has been renamed to 'data_tests'. Please see
2026-06-26T13:56:28.479Z	https://docs.getdbt.com/docs/build/data-tests#new-data_tests-syntax for more
2026-06-26T13:56:28.479Z	information.
2026-06-26T13:56:29.686Z	@[0m13:56:29 [W33mWARNING@0m]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
2026-06-26T13:56:29.686Z	There are 1 unused configuration paths:
2026-06-26T13:56:29.686Z	- models.forecast_project.example
2026-06-26T13:56:30.184Z	@[0m13:56:30 Found 7 models, 5 data tests, 3 sources, 428 macros
2026-06-26T13:56:30.189Z	@[0m13:56:30
2026-06-26T13:56:30.672Z	@[0m13:56:30 Concurrency: 1 threads (target='aws')
2026-06-26T13:56:30.672Z	@[0m13:56:30
2026-06-26T13:56:30.686Z	@[0m13:56:30 1 of 7 START sql view model public.stg_source_1 [RUN]
2026-06-26T13:56:31.064Z	@[0m13:56:31 1 of 7 OK created sql view model public.stg_source_1 [32mCREATE VIEW@0m in 0.37s]
2026-06-26T13:56:31.066Z	@[0m13:56:31 2 of 7 START sql view model public.stg_source_1_stations [RUN]
2026-06-26T13:56:31.176Z	@[0m13:56:31 2 of 7 OK created sql view model public.stg_source_1_stations [32mCREATE VIEW@0m in 0.11s]
2026-06-26T13:56:31.179Z	@[0m13:56:31 3 of 7 START sql view model public.stg_source_2 [RUN]
2026-06-26T13:56:31.466Z	@[0m13:56:31 3 of 7 OK created sql view model public.stg_source_2 [32mCREATE VIEW@0m in 0.29s]
2026-06-26T13:56:31.469Z	@[0m13:56:31 4 of 7 START sql view model public.stg_source_3 [RUN]
2026-06-26T13:56:31.593Z	@[0m13:56:31 4 of 7 OK created sql view model public.stg_source_3 [32mCREATE VIEW@0m in 0.12s]
2026-06-26T13:56:31.596Z	@[0m13:56:31 5 of 7 START sql table model public.dim_weather_stations [RUN]

Retour en haut de page ^

Étape 5 - Déployez sur AWS

Suivre la pipeline avec Aws CloudWatch

But identifier le bon trafic et les erreurs

Gestion des journaux > /ecs/dbt-forecast-task > ecs/dbt-container/96be8ec8e...

Événements de journaux

Vous pouvez utiliser la barre de filtre ci-dessous pour rechercher et faire correspondre des termes, des expressions ou des valeurs dans vos événements de journal. [En savoir plus sur les modèles de filtre](#)

Filter les événements : appuyez sur Entrée pour rechercher. Effacer 1m 30m 1h 12h Personnalisées Fuseau horaire UTC Affichage

Horodatage	Message
2026-06-26T13:56:24.652Z	@[0m13:56:24 Unable to do partial parsing because saved manifest not found. Starting full parse.
2026-06-26T13:56:28.479Z	@[0m13:56:28 [W33mWARNING@0m]: Deprecated functionality
2026-06-26T13:56:28.479Z	The 'tests' config has been renamed to 'data_tests'. Please see
2026-06-26T13:56:28.479Z	https://docs.getdbt.com/docs/build/data-tests#new-data_tests-syntax for more
2026-06-26T13:56:28.479Z	information.
2026-06-26T13:56:29.686Z	@[0m13:56:29 [W33mWARNING@0m]: Configuration paths exist in your dbt_project.yml file which do not apply to any resources.
2026-06-26T13:56:29.686Z	There are 1 unused configuration paths:
2026-06-26T13:56:29.686Z	- models.forecast_project.example
2026-06-26T13:56:30.184Z	@[0m13:56:30 Found 7 models, 5 data tests, 3 sources, 428 macros
2026-06-26T13:56:30.189Z	@[0m13:56:30
2026-06-26T13:56:30.672Z	@[0m13:56:30 Concurrency: 1 threads (target='aws')
2026-06-26T13:56:30.672Z	@[0m13:56:30
2026-06-26T13:56:30.686Z	@[0m13:56:30 1 of 7 START sql view model public.stg_source_1 [RUN]
2026-06-26T13:56:31.064Z	@[0m13:56:31 1 of 7 OK created sql view model public.stg_source_1 [32mCREATE VIEW@0m in 0.37s]
2026-06-26T13:56:31.066Z	@[0m13:56:31 2 of 7 START sql view model public.stg_source_1_stations [RUN]
2026-06-26T13:56:31.176Z	@[0m13:56:31 2 of 7 OK created sql view model public.stg_source_1_stations [32mCREATE VIEW@0m in 0.11s]
2026-06-26T13:56:31.179Z	@[0m13:56:31 3 of 7 START sql view model public.stg_source_2 [RUN]
2026-06-26T13:56:31.466Z	@[0m13:56:31 3 of 7 OK created sql view model public.stg_source_2 [32mCREATE VIEW@0m in 0.29s]
2026-06-26T13:56:31.469Z	@[0m13:56:31 4 of 7 START sql view model public.stg_source_3 [RUN]
2026-06-26T13:56:31.593Z	@[0m13:56:31 4 of 7 OK created sql view model public.stg_source_3 [32mCREATE VIEW@0m in 0.12s]
2026-06-26T13:56:31.596Z	@[0m13:56:31 5 of 7 START sql table model public.dim_weather_stations [RUN]

Retour en haut de page ^

MERCI de m'avoir écouté